

COMPUTER GRAPHICS

Unit-4: Clipping

Comprehensive Study Notes

BCA Semester VI - HPU

Clipping

Introduction to Clipping

Definition: Clipping is the process of removing portions of a graphic primitive (line, polygon, text, etc.) that lie outside a specified region called the clipping window or clipping boundary.

Types of Clipping:

1. **Point Clipping** - Checks if point is inside window using simple comparisons
2. **Line Clipping** - Clips lines against window boundaries (Cohen-Sutherland, Liang-Barsky, Nicholl-Lee-Nicholl)
3. **Polygon Clipping** - Clips polygon against window (Sutherland-Hodgeman, Weiler-Atherton)
4. **Curve Clipping** - Clips curves against window using bounding boxes
5. **Text Clipping** - Clips text characters using various methods

1. Point Clipping

Definition: Point clipping is the simplest form of clipping where we determine whether a point lies inside or outside the clipping window.

Algorithm:

```
if (x ≥ x_min AND x ≤ x_max AND y ≥ y_min AND y ≤ y_max)
    then Point is INSIDE
else
    Point is OUTSIDE
```

Clipping Window:

```
x_min ≤ x ≤ x_max
y_min ≤ y ≤ y_max
```

2. Line Clipping

2.1 Cohen-Sutherland Line Clipping Algorithm

Definition: Cohen-Sutherland is an efficient line clipping algorithm that divides the 2D space into nine regions using a 4-bit code and clips lines using bitwise operations.

Region Codes (Out Codes):

Bit	Position	Condition
Bit 3 (8)	Top	$y > y_{\max}$
Bit 2 (4)	Bottom	$y < y_{\min}$
Bit 1 (2)	Right	$x > x_{\max}$
Bit 0 (1)	Left	$x < x_{\min}$

```

Nine Region Grid:
+-----+
| 1001 (TL) | 1000 (TOP) | 1010 (TR) |
+-----+-----+-----+
| 0001 (LEFT)| 0000 (INSIDE)| 0010 (RIGHT)|
+-----+-----+-----+
| 0101 (BL) | 0100 (BOTTOM)| 0110 (BR) |
+-----+

```

Algorithm Steps:

1. Assign codes to endpoints P1 and P2
2. if code1 = 0 and code2 = 0: Line is completely inside - ACCEPT
3. if code1 & code2 ≠ 0: Line is outside - REJECT
4. else: Clip line against boundaries, Update endpoints, Repeat from step 1

Code Assignment Function:

```

function computeCode(x, y):
    code = 0
    if (x < x_min) code |= 1 // LEFT
    if (x > x_max) code |= 2 // RIGHT
    if (y < y_min) code |= 4 // BOTTOM
    if (y > y_max) code |= 8 // TOP
    return code

```

Line Clipping Example:

Given: Window (0,0) to (4,4)

Line: P1(-1,2) to P2(5,2)

Step 1: Compute codes

P1(-1,2): $x < 0 \Rightarrow$ LEFT (0001)

P2(5,2): $x > 4 \Rightarrow$ RIGHT (0010)

code1 = 0001, code2 = 0010

Step 2: code1 & code2 = 0000 $\Rightarrow$ Not trivial reject

Step 3: Not trivial accept $\Rightarrow$ Need to clip

Step 4: Clip against LEFT boundary ($x = x_{\min}$)

$m = (2-2)/(5-(-1)) = 0/6 = 0$

$y = 2 + 0(0-(-1)) = 2$

New P1 = (0,2), code1 = 0000

Step 5: Clip against RIGHT boundary ($x = x_{\max}$)

$y = 2 + 0(4-(-1)) = 2$

New P2 = (4,2), code2 = 0000

Result: Clipped line from (0,2) to (4,2)

Advantages and Disadvantages:

Advantages	Disadvantages
Simple and efficient	Only works for rectangular clipping windows
Uses bitwise operations (fast)	Can clip multiple times for some lines
Quickly rejects lines outside window	Not suitable for non-rectangular windows
Works for any rectangular window	Can be inefficient for lines with many segments
Easy to implement	Complex for concave polygons
No floating-point operations for trivial cases	

2.2 Liang-Barsky Line Clipping Algorithm

Definition: Liang-Barsky is a line clipping algorithm that uses parametric representation of lines to clip against the window boundaries using a series of inequalities.

Parametric Line Representation:

$$\begin{aligned}x &= x_1 + u \cdot (x_2 - x_1) = x_1 + u \cdot \Delta x \\y &= y_1 + u \cdot (y_2 - y_1) = y_1 + u \cdot \Delta y \\0 &\leq u \leq 1\end{aligned}$$

Clipping Inequalities:

$$\begin{aligned}x_{\min} &\leq x_1 + u \cdot \Delta x \leq x_{\max} \\y_{\min} &\leq y_1 + u \cdot \Delta y \leq y_{\max}\end{aligned}$$

Rearranged: $u \cdot p_k \leq q_k$ where:

$$\begin{aligned}p_1 &= -\Delta x, & q_1 &= x_1 - x_{\min} \\p_2 &= \Delta x, & q_2 &= x_{\max} - x_1 \\p_3 &= -\Delta y, & q_3 &= y_1 - y_{\min} \\p_4 &= \Delta y, & q_4 &= y_{\max} - y_1\end{aligned}$$

Algorithm Steps:

1. Calculate p and q values
2. $u_1 = 0, u_2 = 1$
3. For each k (1 to 4):
 - if $p_k < 0$: $u_1 = \max(u_1, q_k/p_k)$
 - if $p_k > 0$: $u_2 = \min(u_2, q_k/p_k)$
 - if $p_k == 0$ and $q_k < 0$: REJECT line
4. if $u_1 \leq u_2$: ACCEPT line else: REJECT line

Liang-Barsky Example:

Given: Window (0,0) to (4,4)
Line: P1(1,2) to P2(5,3)

Step 1: $\Delta x = 5-1 = 4, \Delta y = 3-2 = 1$

Step 2: p and q values:

$$\begin{aligned}p_1 &= -4, & q_1 &= 1-0 = 1 \\p_2 &= 4, & q_2 &= 4-1 = 3 \\p_3 &= -1, & q_3 &= 2-0 = 2\end{aligned}$$

$p_4 = 1, \quad q_4 = 4 - 2 = 2$

Step 3: $u_1 = 0, \quad u_2 = 1$

$p_1 = -4 < 0: \quad u_1 = \max(0, 1/(-4)) = \max(0, -0.25) = 0$

$p_2 = 4 > 0: \quad u_2 = \min(1, 3/4) = \min(1, 0.75) = 0.75$

$p_3 = -1 < 0: \quad u_1 = \max(0, 2/(-1)) = \max(0, -2) = 0$

$p_4 = 1 > 0: \quad u_2 = \min(0.75, 2/1) = \min(0.75, 2) = 0.75$

Step 4: $u_1 = 0, \quad u_2 = 0.75 \Rightarrow u_1 \leq u_2 \Rightarrow \text{ACCEPT}$

Step 5: Calculate endpoints

$u_1 = 0: \quad P_{\text{start}} = (1, 2)$

$u_2 = 0.75: \quad P_{\text{end}} = (1 + 0.75 \times 4, 2 + 0.75 \times 1) = (4, 2.75)$

Result: Clipped line from (1,2) to (4,2.75)

2.3 Nicholl-Lee-Nicholl (NLN) Line Clipping Algorithm

Definition: Nicholl-Lee-Nicholl (NLN) is a line clipping algorithm that creates a more efficient method by dividing the plane into regions and using direct intersection calculations.

Working Principle:

1. Determine endpoint regions (3x3 grid)
2. Create L-, T-, and R- regions based on endpoint location
3. Clip line against boundaries using precomputed intersection points
4. More efficient than Cohen-Sutherland

Line Clipping Algorithms Comparison:

Feature	Cohen-Sutherland	Liang-Barsky	Nicholl-Lee-Nicholl
Type	Region Code	Parametric	Region + Parametric
Operations	Bitwise	Floating-point	Mixed
Speed	Fast	Fast	Very Fast
Clipping Iterations	Multiple	Single	Single
Complexity	Simple	Medium	Complex
Best Use	General Purpose	Special Cases	High-speed Applications

3. Polygon Clipping

3.1 Sutherland-Hodgeman Polygon Clipping Algorithm

Definition: Sutherland-Hodgeman is a polygon clipping algorithm that clips a polygon against each edge of the clipping window sequentially.

Working Principle:

- Processes polygon against one boundary at a time
- Four boundaries: Left, Right, Bottom, Top
- Input polygon vertices processed sequentially
- Output becomes input for next boundary

Cases for Each Edge:

Case	Previous (S)	Current (P)	Output
1	Inside	Inside	Output P
2	Inside	Outside	Output Intersection of SP with edge
3	Outside	Inside	Output Intersection, then P
4	Outside	Outside	Output nothing

Polygon Clipping Example:

Given: Rectangle Window (0,0) to (4,4)
 Polygon: Triangle A(1,1), B(5,1), C(3,5)

Clip against Left Boundary ($x = 0$):

A(1,1) IN, B(5,1) IN $\Rightarrow$ Output A, B

B(5,1) IN, C(3,5) IN $\Rightarrow$ Output B, C

C(3,5) IN, A(1,1) IN $\Rightarrow$ Output C, A

Result: A(1,1), B(5,1), C(3,5)

Clip against Right Boundary ($x = 4$):

A(1,1) IN, B(5,1) OUT $\Rightarrow$ Output Intersection I1(4,1)

B(5,1) OUT, C(3,5) IN $\Rightarrow$ Output I1, C(3,5)

C(3,5) IN, A(1,1) IN $\Rightarrow$ Output C, A(1,1)

Result: I1(4,1), C(3,5), A(1,1)

Clip against Bottom Boundary ($y = 0$):

All points have $y \geq 0 \Rightarrow$ No clipping needed

Result: I1(4,1), C(3,5), A(1,1)

```
Clip against Top Boundary (y = 4):  
I1(4,1) IN, C(3,5) OUT => Output Intersection I2(3,4)  
C(3,5) OUT, A(1,1) IN => Output I2, A(1,1)  
A(1,1) IN, I1(4,1) IN => Output A, I1  
  
Final Clipped Polygon: I2(3,4), A(1,1), I1(4,1)
```

Advantages and Disadvantages:

Advantages	Disadvantages
Simple to implement	Only works for convex polygons
Works with any convex polygon	May produce degenerate polygons
Handles polygon vertices efficiently	Cannot handle holes in polygons
Can be extended to 3D	Multiple passes needed
Output is always a polygon	Not suitable for concave polygons

3.2 Weiler-Atherton Polygon Clipping Algorithm

Definition: Weiler-Atherton is a polygon clipping algorithm that can handle both convex and concave polygons, including polygons with holes, by using vertex lists and traversal.

Working Principle:

- Supports concave polygons and holes
- Uses two vertex lists (subject and clip polygons)
- Traversal of intersection points
- Follows boundaries alternately

Algorithm Steps:

1. Find all intersections between subject and clip polygons
2. Mark intersections as entering/exiting
3. Build linked lists of vertices for both polygons
4. Traverse to create clipped polygon
5. Follow entering intersections with subject polygon, exiting with clip polygon

Sutherland-Hodgeman vs Weiler-Atherton Comparison:

Feature	Sutherland-Hodgeman	Weiler-Atherton
Polygon Type	Only convex polygons	Handles any polygon
Implementation	Simple	Complex
Execution Speed	Faster	Slower
Holes	Not supported	Supported
Clipping Method	Sequential clipping	Direct clipping
Memory Required	Less	More
Applications	General purpose	Special purpose

4. Curve Clipping

Definition: Curve clipping is the process of removing parts of curves (circles, ellipses, splines, Bezier curves) that lie outside the clipping window.

Methods for Curve Clipping:

1. **Bounding Box Method** - Check if curve's bounding box intersects window; if no intersection → REJECT; if fully inside → ACCEPT; otherwise → clip further
2. **Parametric Clipping** - Use parametric equation of curve, test parameter ranges, clip against window boundaries
3. **Subdivision Method** - Subdivide curve into segments, test each segment against window, recursively clip segments

5. Text Clipping

Definition: Text clipping is the process of determining which parts of text characters are visible within the clipping window.

Types of Text Clipping:

Type	Description	Accuracy	Speed
All-or-None	Entire string is either displayed or hidden based on bounding box	Low	Fast
Partial	Individual characters checked; inside displayed, outside clipped	Medium	Medium
Character	Individual pixels of characters checked against window	High	Slow

Example:

```
Text: "COMPUTER GRAPHICS"  
Window size for display: limited width  
  
All-or-none: "COMPUTER GRAPHICS" fully visible or fully clipped  
Partial:     "COMPUTER GR" displayed, "APHICS" clipped
```

Quick Reference Summary - Unit IV

Key Terms:

1. **Clipping** - Removing parts of objects outside a region
2. **Clipping Window** - Rectangular region defining visible area
3. **Out Code** - 4-bit code for Cohen-Sutherland algorithm
4. **Parametric Clipping** - Using parameter t for line clipping
5. **Polygon Clipping** - Clipping polygon primitives
6. **Convex Polygon** - All interior angles $< 180^\circ$
7. **Concave Polygon** - Some interior angles $> 180^\circ$
8. **Bounding Box** - Rectangle enclosing an object
9. **Text Clipping** - Clipping text characters

Algorithm Quick Reference:

Algorithm	Type	Complexity	Best For
Point Clipping	Point	$O(1)$	Simple inside test
Cohen-Sutherland	Line	$O(n)$	General purpose
Liang-Barsky	Line	$O(1)$	Efficient clipping
Nicholl-Lee-Nicholl	Line	$O(1)$	High-speed applications
Sutherland-Hodgeman	Polygon	$O(n)$	Convex polygons
Weiler-Atherton	Polygon	$O(n \times m)$	Complex polygons
Curve Clipping	Curve	$O(n)$	Curved objects
Text Clipping	Text	$O(n)$	Text rendering

Review Questions - Unit IV

1. What is clipping? Explain the different types of clipping in computer graphics.
2. Explain the Cohen-Sutherland line clipping algorithm with a suitable example.
3. What are region codes? How are they assigned in the Cohen-Sutherland algorithm?
4. Compare and contrast Cohen-Sutherland and Liang-Barsky line clipping algorithms.
5. Explain the Liang-Barsky line clipping algorithm with a step-by-step example.
6. Describe the Sutherland-Hodgeman polygon clipping algorithm. Explain the four cases for vertex processing.
7. What are the advantages and limitations of the Sutherland-Hodgeman polygon clipping algorithm?
8. Explain the Weiler-Atherton polygon clipping algorithm. How is it different from Sutherland-Hodgeman?
9. Write a short note on curve clipping and its methods.
10. Explain the three types of text clipping with examples.
11. Compare point clipping and line clipping with respect to complexity and use cases.
12. What is the role of clipping in computer graphics? Give real-world applications.